rutalan

GESTIÓN DE CRÉDITOS

ESTÁNDARES DE CODIFICACIÓN

Convenciones, buenas prácticas y reglas de calidad del proyecto Rutalan

Backend PHP · Frontend Angular · Base de datos MySQL · Seguridad

Autores:
Saul Fragua Nova
Harol Sebastián Moreno Medina

Análisis y Desarrollo de Software -ADSO
Ficha 2977414 - Etapa práctica
Servicio Nacional de Aprendizaje – SENA
Regional Dosquebradas - Risaralda

Versión 1.0
Octubre de 2026

Contenido

1. Introducción 4
1.1 Propósito 4
1.2 Alcance 4
1.3 Cómo leer este documento 4
2. Principios generales 5
2.1 Idioma 5
2.2 Formato común (todos los lenguajes) 5
3. Control de versiones 7
3.1 Ramas 7
3.2 Mensajes de commit 7
3.3 Qué NO se versiona 7
3.4 Pull Requests 8
4. Estándares de PHP (backend) 9
4.1 Arquitectura 9
4.2 Nombres 9
4.3 Formato 9
4.4 Estructura de un controlador 10
4.5 Estructura de un modelo 10
4.6 Acceso a datos (reglas obligatorias) 11
4.7 Autenticación y autorización 11
4.8 Peticiones, respuestas y errores 12
4.9 Carga de archivos 12
4.10 Configuración y secretos 12
5. Estándares de TypeScript y Angular (frontend) 14
5.1 Estructura de carpetas 14
5.2 Nombres 14
5.3 Formato (Prettier y EditorConfig) 14
5.4 TypeScript 15
5.5 Servicios y llamadas HTTP 15
5.6 Componentes y plantillas 16
5.7 Mensajes y manejo de errores 16
5.8 Almacenamiento en el navegador 16
6. Estilos, diseño y accesibilidad 18
6.1 CSS 18
6.2 Diseño responsivo 18

6.3 Accesibilidad 18
7. Estándares de base de datos 19
7.1 Nombres 19
7.2 Reglas 19
8. Seguridad y protección de datos personales 20
8.1 Marco legal 20
8.2 Lista de verificación de seguridad 20
8.3 Datos personales en el desarrollo 20
9. Pruebas y calidad 21
9.1 Verificaciones obligatorias antes del Pull Request 21
9.2 Qué se prueba 21
9.3 Criterios de aceptación de un cambio 21
10. Revisión de código y documentación 22
10.1 Lista de verificación para el revisor 22
10.2 Comentarios y documentación 22
10.3 Herramientas recomendadas 22
11. Deuda técnica conocida 24
12. Anexos 25
12.1 Resumen de convenciones de nombres 25
12.2 Lista de verificación rápida del Pull Request 25
12.3 Control de versiones del documento 25
12.4 Documentos relacionados 26

1. Introducción

1.1 Propósito

Este documento define los estándares de codificación de **Rutalan**: cómo se nombran, estructuran, escriben, revisan y prueban los cambios en el código. Su objetivo es que cualquier persona del equipo pueda leer, mantener y ampliar el proyecto con el mismo criterio, reduciendo errores y deuda técnica.

1.2 Alcance

Componente	Ubicación	Tecnología
Backend (API)	backend/	PHP 8, PDO, patrón MVC, MySQL/MariaDB
Frontend	frontend/	Angular 21 (NgModules), TypeScript estricto, Tailwind CSS 4
Servicio de WhatsApp	whatsapp-api/	Node.js, Express, whatsapp-web.js
Landing page	landing/	HTML, CSS y JavaScript estático
Base de datos	backend/config/rutalan.sql	MySQL / MariaDB

1.3 Cómo leer este documento

Las reglas usan un vocabulario normativo:

Término	Significado
DEBE / NO DEBE	Regla obligatoria. Un cambio que la incumpla no se aprueba.
DEBERÍA / NO DEBERÍA	Recomendación fuerte. Solo se omite con una razón justificada en el Pull Request.
PUEDE	Opcional, según el criterio de quien desarrolla.

> **NOTA:** Donde el código existente no cumple una regla, se indica en la sección 11 (Deuda técnica conocida). Las reglas aplican a **código nuevo y a cualquier archivo que se modifique**; no es obligatorio reescribir lo antiguo en el mismo cambio.

2. Principios generales

- **Seguridad primero.** Ninguna regla de negocio ni de permisos puede depender solo del frontend. La API valida sesión, rol y datos de entrada.
- **Datos personales protegidos.** El sistema maneja datos y fotos de cédulas de clientes y fiadores (Ley 1581 de 2012). Nunca se versionan, registran en logs ni se muestran sin necesidad.
- **Legibilidad sobre ingenio.** Se prefiere el código simple y explícito. Si algo requiere un truco, se documenta el motivo.
- **Cambios pequeños y revisables.** Un cambio resuelve un propósito. Los cambios masivos de formato no se mezclan con cambios de lógica.
- **Consistencia.** Se sigue la convención existente en el archivo y el módulo; si hay conflicto, manda este documento.
- **No repetir.** La lógica compartida va en un servicio, modelo o función reutilizable, no copiada.

2.1 Idioma

Elemento	Idioma	Ejemplo
Nombres del dominio (entidades, acciones, columnas)	Español	consultarPorId, listaGastos, id_ruta
Palabras técnicas del lenguaje o framework	Inglés	Observable, Injectable, FormData
Comentarios y documentación	Español	// Valida que la caja esté abierta
Textos para el usuario	Español (Colombia)	"Pago registrado correctamente"
Mensajes de commit	Español (o inglés), sin mezclar	fix(caja): corrige el total recolectado

No se usan tildes ni la letra "ñ" en identificadores de código (variables, métodos, clases, archivos, columnas), solo en textos y comentarios.

2.2 Formato común (todos los lenguajes)

- Codificación **UTF-8**, sin BOM. Salto de línea final en cada archivo y sin espacios al final de las líneas (configurado en `.editorconfig`).
- Indentación con **espacios**, nunca tabuladores: 4 en PHP y 2 en TypeScript, HTML, CSS y JSON.
- Longitud de línea recomendada: **100 caracteres** (Prettier en el frontend).

- Un archivo, una responsabilidad. Los archivos de más de ~500 líneas DEBERÍAN dividirse (hoy existen componentes de más de 1000 líneas; ver sección 11).
- No se deja código comentado ni `TODO` sin responsable o fecha. El historial de Git conserva el código antiguo.

3. Control de versiones

3.1 Ramas

El desarrollo parte de `main`. Las ramas de trabajo siguen el formato `tipo/descripcion-corta`:

Prefijo	Uso	Ejemplo
feature/	Nueva funcionalidad	feature/exportar-informe-gastos
fix/	Corrección de errores	fix/saldo-negativo-en-caja
docs/	Solo documentación	docs/manual-usuario
refactor/	Mejora interna sin cambio de comportamiento	refactor/servicio-creditos
chore/	Mantenimiento, dependencias, configuración	chore/actualizar-angular

3.2 Mensajes de commit

Se usa **Conventional Commits**:

Formato

```
tipo(ámbito opcional): descripción corta en imperativo

Cuerpo opcional: qué cambió y por qué (no el cómo).
```

- Tipos permitidos: `feat`, `fix`, `docs`, `style`, `refactor`, `perf`, `test`, `chore` y `security`.
- Primera línea de **máximo 72 caracteres**, sin punto final.
- Un commit debe dejar el proyecto compilando y funcionando.
- Los mensajes genéricos ("cambios", "arreglos", "update") NO DEBEN usarse.

3.3 Qué NO se versiona

El `.gitignore` de la raíz DEBE mantenerse y protege lo siguiente. Si algo de esta lista aparece en un `git status`, se corrige antes del commit:

Archivo o carpeta	Motivo
backend/config/key.php	Claves de reCAPTCHA, credenciales de base de datos y AUTH_SECRET.
backend/config/.auth_secret	Secreto de firma de los tokens de sesión.

Archivo o carpeta	Motivo
backend/uploads/*	Fotos de clientes y cédulas: datos personales (Ley 1581 de 2012).
.env, .env.*	Variables de entorno (se permite .env.example).
node_modules/, dist/	Dependencias y artefactos de compilación.
*.rar, *.bak, backups/, *.log	Comprimidos, respaldos y registros.
whatsapp-api/.wwebjs_auth, .wwebjs_cache	Sesión y caché de WhatsApp Web.
graphify-out/	Salida generada de análisis del código.

> **IMPORTANTE:** Si un secreto o un dato personal llega a un commit, **borrarlo en un commit posterior no es suficiente**: queda en el historial. Debe avisarse de inmediato, rotar el secreto y limpiar el historial.

3.4 Pull Requests

- Todo cambio llega a `main` mediante Pull Request con al menos **una aprobación**.
- El PR explica qué resuelve, cómo se probó e incluye capturas si cambia la interfaz (con datos personales ocultos).
- No se fusiona con la compilación rota ni con comentarios de revisión sin resolver.

4. Estándares de PHP (backend)

4.1 Arquitectura

El backend sigue el patrón **MVC** simplificado. Cada módulo tiene un controlador y un modelo:

Capa	Carpeta	Responsabilidad
Controlador	backend/controllers/	Recibe la petición HTTP, autentica, valida los datos y responde en JSON. No contiene SQL.
Modelo	backend/models/	Ejecuta las consultas a la base de datos y la lógica de negocio. No lee `$_GET`, `$_POST` ni envía respuestas.
Configuración	backend/config/	Conexión (`conexion.php`), secretos (`key.php`), autenticación (`auth.php`).
Servicios	backend/services/	Integraciones externas, por ejemplo WhatsApp.

4.2 Nombres

Elemento	Convención	Ejemplo
Archivo de controlador	camelCase + "Controlador.php"	clientesControlador.php
Archivo de modelo	camelCase + "Modelos.php"	creditosModelos.php
Clases	PascalCase	Creditos, PlanPagos, ClavesCobrador
Métodos y funciones	camelCase, verbo en español	consultarPorId(), insertar(), calcularSeguro()
Variables y parámetros	camelCase	$idCredito, $montoCredito
Constantes	MAYÚSCULAS_CON_GUIÓN_BAJO	AUTH_TTL, RECAPTCHA_SECRET
Acciones de la API (`?control=`)	camelCase, igual al método del modelo	consultarPorId, actualizarOrdenCobranza

Verbos estándar para métodos de modelo: `consultar`, `consultarPorId`, `filtrar`/`buscar`, `insertar`, `editar`, `eliminar`, `activar`/`inactivar`.

4.3 Formato

- Indentación de **4 espacios**; llave de apertura en la misma línea; una instrucción por línea (PSR-12 como guía).

- Los archivos que solo contienen PHP NO DEBERÍAN cerrar con `?>` (evita salidas accidentales por espacios al final).
- Para incluir archivos se usa `__DIR__` (por ejemplo `require_once __DIR__ . "/../config/auth.php"`) en lugar de rutas relativas al directorio de trabajo.
- Se declara visibilidad (`public`, `private`) en todos los métodos y propiedades.
- Cada método público DEBERÍA llevar un comentario breve (PHPDoc) con el propósito, parámetros y retorno.

4.4 Estructura de un controlador

backend/controllers/ejemploControlador.php

```
<?php
header("Access-Control-Allow-Origin: *");   // restringir al dominio en producción

require_once "../config/conexion.php";
require_once "../models/ejemploModelos.php";
require_once __DIR__ . '/../config/auth.php';
requerirAuth();                              // 401 sin token, 403 sin permiso

$control = $_GET['control'] ?? '';
$ejemplo = new Ejemplo($conexion);

switch ($control) {
    case 'consultar':
        echo json_encode($ejemplo->consultar());
        break;

    case 'insertar':
        if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
            http_response_code(405);
            echo json_encode(['estado' => 'error', 'mensaje' => 'Use POST']);
            break;
        }
        $params = json_decode(file_get_contents('php://input'), true);
        if (empty($params['nombre'])) {
            http_response_code(400);
            echo json_encode(['estado' => 'error', 'mensaje' => 'El nombre es obligatorio']);
            break;
        }
        echo json_encode($ejemplo->insertar($params));
        break;

    default:
        http_response_code(400);
        echo json_encode(['estado' => 'error', 'mensaje' => 'Control no válido']);
}
```

4.5 Estructura de un modelo

backend/models/ejemploModelos.php

```php
<?php
class Ejemplo {
    private $conexion;

    public function __construct($conexion) {
        $this->conexion = $conexion;
    }

    // Consulta un registro por su identificador
    public function consultarPorId($id) {
        $sql = "SELECT id_ejemplo, nombre FROM ejemplos WHERE id_ejemplo = :id";
        $stmt = $this->conexion->prepare($sql);
        $stmt->execute([':id' => $id]);
        return $stmt->fetch(PDO::FETCH_ASSOC);
    }
}
```

4.6 Acceso a datos (reglas obligatorias)

- **DEBE** usarse siempre **PDO con sentencias preparadas** y parámetros enlazados. NO DEBE concatenarse ni interpolarse una variable dentro de una cadena SQL.
- Se selecciona solo lo necesario: NO DEBE usarse `SELECT *` en consultas que viajan al cliente, y **nunca** se devuelve la columna de contraseña.
- Las operaciones que modifican varias tablas DEBEN ejecutarse dentro de una **transacción** (`beginTransaction`, `commit`, `rollBack`).
- Los valores de dinero se almacenan como `DECIMAL`, nunca como `FLOAT`. Los cálculos se hacen con cuidado de redondeo y se prueban.
- Las fechas se guardan en formato `Y-m-d` / `Y-m-d H:i:s`. La zona horaria del sistema es **America/Bogota** (definida en `config.php` y `conexion.php`).
- La conexión configura `PDO::ERRMODE_EXCEPTION`; los errores de base de datos se capturan en el controlador.

4.7 Autenticación y autorización

- Todo controlador, **excepto** **`loginControlador.php`**, DEBE invocar `requerirAuth()` antes de leer cualquier dato.
- Las acciones que solo puede ejecutar un administrador DEBEN registrarse en la constante `ACCIONES_SOLO_ADMIN` de `backend/config/auth.php`. NO se valida el rol únicamente en el frontend.
- Los tokens se firman con HMAC-SHA256 y expiran a las 12 horas (`AUTH_TTL`). El secreto (`AUTH_SECRET`) vive en `key.php` o en `config/.auth_secret`, nunca en el código.
- Un usuario puede ejecutar una acción sobre un recurso solo si le pertenece (por ejemplo, un cobrador sobre sus rutas y su caja). Esa validación se hace en el servidor.

> **REGLA:** Una acción nueva que modifique datos sin pasar por `requerirAuth()` y sin validar los parámetros **no se aprueba**.

4.8 Peticiones, respuestas y errores

- **Métodos HTTP:** consultar usa `GET`; crear, editar, eliminar, activar, cambiar estado y registrar pagos o movimientos DEBEN usar `POST` (o `PUT`/`DELETE`). Las acciones que modifican datos NO DEBEN ejecutarse por `GET`.
- **Formato de respuesta:** JSON con la clave `estado` (`ok` o `error`) y `mensaje` legible. En código nuevo se usa `estado`; el uso histórico de `resultado` se mantiene solo por compatibilidad.
- **Códigos HTTP:** 200 correcto, 400 datos inválidos, 401 sin sesión, 403 sin permiso, 404 no encontrado, 405 método no permitido, 500 error interno.
- **Errores:** el detalle técnico de una excepción se registra con `error_log()`. Al cliente se le devuelve un mensaje genérico; NO se expone `$e->getMessage()`, rutas de archivos ni consultas SQL.
- **Validación:** toda entrada (`$_GET`, `$_POST`, cuerpo JSON, archivos) se valida (existencia, tipo, longitud, rango) antes de usarla.
- **Salida:** `ini_set('display_errors', 0)` en producción; los errores solo van al registro.

4.9 Carga de archivos

- Se valida la extensión **y** el tipo MIME real (`finfo`) permitiendo solo `jpg`, `jpeg`, `png` y `webp`; se limita el tamaño máximo.
- El nombre final lo genera el servidor (no se confía en el nombre original) y se guarda en `backend/uploads/<tipo>/<id>/`.
- Los permisos de carpeta NO DEBEN ser `0777` en producción (usar `0755`).
- Las imágenes de cédulas DEBERÍAN servirse mediante un endpoint que verifique el token en lugar de como archivos estáticos públicos.

4.10 Configuración y secretos

- Los secretos van en `backend/config/key.php` (ignorado por Git). La plantilla `key_example.php` se mantiene actualizada con cada constante nueva, **sin valores reales**.
- En desarrollo la base de datos usa `root` sin contraseña en `localhost`; en producción las credenciales provienen de `key.php`.

- CORS: `Access-Control-Allow-Origin: *` es aceptable solo en desarrollo. En producción DEBERÍA limitarse al dominio de la aplicación.

5. Estándares de TypeScript y Angular (frontend)

5.1 Estructura de carpetas

frontend/src/app/

```
app/
├── estructura/      # Navbar, sidebar, footer y layout principal
├── guards/          # AdminGuard e interceptor de autenticación
├── modulos/         # Una carpeta por pantalla: clientes/, creditos/, cobros/...
│   └── clientes/
│       ├── clientes.ts      # Componente (lógica)
│       ├── clientes.html    # Plantilla
│       └── clientes.css     # Estilos del componente
├── servicios/       # Un servicio por controlador de la API (cliente.ts, caja.ts...)
├── app-module.ts    # Declaración de componentes (NgModule)
└── app-routing-module.ts
```

El proyecto usa **NgModules** (`standalone: false`). Todo componente nuevo se declara en `app-module.ts` y su ruta en `app-routing-module.ts`.

5.2 Nombres

Elemento	Convención	Ejemplo
Archivos	kebab-case	gestion-pago.ts, claves-cobrador.ts
Clases (componentes, servicios, guards)	PascalCase	GestionPago, AdminGuard
Selector de componente	prefijo `app-` en kebab-case	app-gastos
Propiedades y métodos	camelCase en español	listaGastos, abrirModalFoto()
Booleanos	prefijo es/tiene/mostrar/puede	tieneCajaAbierta, mostrarModal
Constantes globales	MAYÚSCULAS o camelCase en `environment`	apiUrl, whatsappApiUrl
Observables	sufijo `$` (opcional)	clientes$
Interfaces y tipos	PascalCase sin prefijo `I`	Cliente, Credito

5.3 Formato (Prettier y EditorConfig)

frontend/package.json (extracto)

```
"prettier": {
  "printWidth": 100,
  "singleQuote": true,
  "overrides": [{ "files": "*.html", "options": { "parser": "angular" } }]
}
```

- Indentación de 2 espacios, comillas simples en TypeScript, punto y coma al final de cada instrucción.
- Orden de miembros en una clase: propiedades públicas, propiedades privadas, constructor, ciclo de vida (`ngOnInit`, `ngOnDestroy`), métodos públicos y métodos privados.
- Orden de imports: Angular, terceros, servicios y modelos propios.
- Antes de integrar, el código DEBE formatearse con `npx prettier --write` sobre los archivos modificados.

5.4 TypeScript

- `strict` está activado en `tsconfig.json`: NO DEBE desactivarse ni silenciarse con `@ts-ignore`.
- En código nuevo NO DEBE usarse `any`. Se definen **interfaces** para los datos de la API (por ejemplo `Cliente`, `Credito`, `Pago`) y se tipan los servicios. Cuando el tipo es realmente desconocido se usa `unknown`.
- Se usan `const` por defecto y `let` solo cuando hay reasignación; nunca `var`.
- Se prefieren funciones pequeñas con una responsabilidad. Un método de más de ~60 líneas DEBERÍA dividirse.
- Los valores mágicos (tasas, topes, días, porcentajes) se declaran como constantes con nombre, por ejemplo `TASA_USURA_MAXIMA`.

5.5 Servicios y llamadas HTTP

- Toda llamada a la API DEBE estar en un **servicio** de `src/app/servicios/`, nunca en una plantilla ni directamente en un componente con `fetch`.
- La URL base sale de `environment.apiUrl`; no se escriben direcciones fijas (`localhost`) en el código.
- El token de sesión lo agrega el **interceptor** `auth.interceptor.ts`; los servicios no manejan cabeceras de autenticación.
- Las acciones que modifican datos usan `post` (ver sección 4.8).

Ejemplo de servicio tipado

```
@Injectable({ providedIn: 'root' })
export class Clientes {
  private url = `${environment.apiUrl}/controllers/clientesControlador.php`;

  constructor(private http: HttpClient) {}
```

```
  consultarPorId(id: number) {
    return this.http.get<Cliente>(`${this.url}?control=consultarPorId&id=${id}`);
  }

  insertar(datos: NuevoCliente) {
    return this.http.post<RespuestaApi>(`${this.url}?control=insertar`, datos);
  }
}
```

5.6 Componentes y plantillas

- El componente coordina la pantalla; la lógica de negocio compartida va en servicios.
- Las plantillas usan el control de flujo moderno (`@if`, `@for` con `track`) y NO contienen cálculos pesados ni llamadas a métodos costosos dentro de la interpolación.
- Las suscripciones a observables DEBEN cancelarse en `ngOnDestroy` (o usar `async` / `takeUntilDestroyed`).
- El acceso a `window`, `document` y `localStorage` DEBE protegerse con `isPlatformBrowser`, porque la aplicación admite renderizado en servidor.
- Los formularios validan los campos obligatorios y muestran un mensaje claro por campo; la validación del cliente **no sustituye** la del servidor.
- Los permisos se aplican en el servidor. El `AdminGuard` y el menú por rol solo mejoran la experiencia de navegación.

5.7 Mensajes y manejo de errores

- Todo error de una llamada HTTP se captura y se informa al usuario con un mensaje comprensible en español.
- Un `401` cierra la sesión local y redirige al login (lo hace el interceptor); un `403` informa que no tiene permisos.
- NO DEBEN quedar `console.log` en el código integrado; para depurar se usan puntos de interrupción. Los errores inesperados pueden usar `console.error`.
- Para avisos al usuario en código nuevo se prefieren componentes de notificación o modales en lugar de `alert()` del navegador.

5.8 Almacenamiento en el navegador

- `localStorage` guarda únicamente `token` y `usuario` (datos mínimos de sesión: id, nombre, rol, rutas asignadas).
- NO se almacenan contraseñas, datos de clientes, fotos ni información financiera en el navegador.

- Al cerrar sesión se eliminan `usuario` y `token`.

6. Estilos, diseño y accesibilidad

6.1 CSS

- Los estilos de una pantalla van en el `.css` de su componente. Los estilos compartidos están en `design-system.css`, `global-styles.css` y `responsive.css`; se reutilizan antes de crear nuevos.
- Se usa Tailwind CSS 4 para utilidades; no se mezclan estilos en línea (`style=""`) con clases cuando existe una clase equivalente.
- Los colores, sombras y radios de la marca se toman de las variables del sistema de diseño, no se repiten valores hexadecimales sueltos.
- Los selectores son específicos del componente; no se usa `!important` salvo para sobreescribir librerías externas (documentando el motivo).

6.2 Diseño responsivo

- Toda pantalla DEBE funcionar en **móvil, tableta y escritorio**. Se diseña primero para móvil (*mobile-first*) y se amplía con *media queries*.
- Puntos de quiebre utilizados: **576 px, 768 px, 992 px y 1200 px**.
- No debe aparecer desplazamiento horizontal de la página. Las tablas anchas van dentro de un contenedor con desplazamiento propio.
- Los elementos táctiles (botones, enlaces) DEBEN ser lo bastante grandes para tocarse con el dedo.

6.3 Accesibilidad

- Todo campo de formulario tiene su `<label>` asociado y los botones de solo icono tienen `title` o `aria-label`.
- El contraste de texto y fondo DEBE ser suficiente para leerse (relación mínima 4.5:1).
- Se respeta `prefers-reduced-motion`: las animaciones se desactivan o reducen cuando el usuario lo solicita.
- No se transmite información solo con el color (por ejemplo, el estado también lleva texto).

7. Estándares de base de datos

7.1 Nombres

Elemento	Convención	Ejemplo
Tablas	snake_case en minúsculas, en plural	clientes, movimientos_caja, usuario_ruta
Columnas	snake_case en minúsculas	fecha_registro, foto_cedula_frontal
Llave primaria	`id_` + nombre de la entidad	id_cliente, id_credito
Llave foránea	mismo nombre que la llave primaria referenciada	id_ruta, id_usuario
Índices	`idx_tabla_columna`	idx_pagos_id_credito
Estados	`TINYINT(1)` (0/1) o ENUM documentado	estado, activo

7.2 Reglas

- Toda tabla tiene llave primaria. Las relaciones se declaran con llaves foráneas e índices en las columnas de búsqueda frecuente.
- Montos y tasas usan `DECIMAL(p,s)`; fechas usan `DATE`, `DATETIME` o `TIMESTAMP`; el juego de caracteres es `utf8mb4`.
- Las contraseñas se guardan **solo con hash** (`password_hash`). Nunca se almacenan ni se devuelven en texto plano.
- Todo cambio de estructura se refleja en `backend/config/rutalan.sql` en el mismo Pull Request y se describe en el PR (tablas, columnas, migración de datos existentes).
- `rutalan.sql` contiene únicamente estructura y datos mínimos de prueba. **Nunca** datos reales de clientes.
- Antes de un cambio de estructura en producción se toma un respaldo y se prueba el script en una copia.
- Se prefiere el borrado lógico (columna `activo`/`estado`) al borrado físico en entidades con historial financiero.

8. Seguridad y protección de datos personales

8.1 Marco legal

Rutalan trata datos personales y documentos de identidad de clientes y fiadores. Esto implica obligaciones de la **Ley 1581 de 2012** y sus decretos reglamentarios: finalidad del tratamiento, autorización del titular, seguridad y confidencialidad de la información.

8.2 Lista de verificación de seguridad

Control	Regla
Autenticación	Toda acción de la API, salvo el login, exige token válido (`requerirAuth()`).
Autorización	El rol se valida en el servidor (`ACCIONES_SOLO_ADMIN`) y el acceso a recursos propios se verifica por usuario.
Inyección SQL	Solo sentencias preparadas; ninguna variable dentro del SQL.
XSS	No se inserta HTML sin sanear (`innerHTML`, `bypassSecurityTrust*`). Se usa la interpolación de Angular.
Contraseñas	Hash con `password_hash`; nunca en respuestas ni en logs.
Secretos	Fuera del repositorio (`key.php`, `.auth_secret`). Se rotan si se exponen.
Errores	Mensaje genérico al usuario; detalle en `error_log`.
Archivos subidos	Validación de tipo, tamaño y nombre; fuera del control de versiones.
Sesión	Token con vencimiento (12 h). Un 401 cierra la sesión local.
CORS	Restringir al dominio de la aplicación en producción.
Bots	reCAPTCHA obligatorio en el login.
Dependencias	Se revisan advertencias de `npm audit` y se actualizan paquetes con vulnerabilidades.

8.3 Datos personales en el desarrollo

- En desarrollo y pruebas se usan **datos ficticios**. No se copian bases de producción a equipos locales.
- Las capturas de pantalla para documentación, issues o Pull Requests DEBEN ocultar nombres, documentos, teléfonos, direcciones, fotos y montos reales.
- Los registros (`error_log`, consola) NO DEBEN contener datos personales completos.
- Si se detecta una fuga de datos o de secretos, se informa de inmediato a los mantenedores para contenerla.

9. Pruebas y calidad

9.1 Verificaciones obligatorias antes del Pull Request

Verificación	Comando o acción
Compila el frontend	cd frontend && ng build
Pruebas unitarias del frontend	cd frontend && ng test
Sintaxis PHP	php -l ruta/al/archivo.php (todos los archivos modificados)
Formato	npx prettier --check sobre los archivos modificados
Prueba manual	Recorrer el flujo afectado como administrador y como cobrador
Responsive	Probar en móvil, tableta y escritorio
Casos de error	Datos vacíos o inválidos, sin sesión, sin permisos, sin caja abierta

9.2 Qué se prueba

- La **lógica financiera** (cálculo de seguro, intereses, cuotas, saldos, cierres de caja) DEBE tener pruebas, con casos límite y redondeos.
- Los **permisos**: una acción exclusiva de administrador debe responder `403` con un token de cobrador y `401` sin token.
- Los **servicios** del frontend con pruebas unitarias sobre las llamadas y transformaciones de datos.
- Cada error corregido DEBERÍA incluir una prueba (o un caso documentado en el PR) que lo reproduzca.

> **NOTA:** Hoy la cobertura de pruebas automatizadas es mínima. Se recomienda incorporar PHPUnit para el backend y ampliar las pruebas de Angular de forma progresiva, comenzando por la lógica de créditos, pagos y cajas.

9.3 Criterios de aceptación de un cambio

- Cumple el requisito pedido y no rompe otros módulos.
- Compila sin errores ni advertencias nuevas.
- No introduce riesgos de seguridad ni expone datos personales.
- Mantiene o mejora la legibilidad y está documentado cuando corresponde.

10. Revisión de código y documentación

10.1 Lista de verificación para el revisor

Área	Pregunta
Seguridad	¿La acción pasa por `requerirAuth()` y valida el rol y la entrada? ¿Hay SQL concatenado, secretos o datos personales en el diff?
Correctitud	¿Hace lo que dice el PR? ¿Maneja casos límite y errores?
Datos	¿Las operaciones de varias tablas usan transacción? ¿Los montos usan DECIMAL?
Diseño	¿Respeta MVC y la estructura de carpetas? ¿Evita duplicar lógica?
Estilo	¿Sigue las convenciones de nombres y el formato (Prettier/EditorConfig)?
Interfaz	¿Funciona en móvil y escritorio? ¿Los mensajes son claros?
Pruebas	¿Se probó con ambos roles y los casos de error?
Documentación	¿Se actualizaron README, manual de usuario o estos estándares si aplica?

10.2 Comentarios y documentación

- Los comentarios explican el **porqué** de una decisión, no repiten lo que el código ya dice.
- Las funciones públicas con lógica no obvia llevan un comentario de cabecera (PHPDoc / JSDoc) con propósito, parámetros y retorno.
- Las reglas de negocio (por ejemplo, el seguro de $5 por cada $100, el tope de usura o la regla de plazos de 40 y 70 días) se documentan junto al código que las aplica.
- Todo cambio visible para el usuario actualiza el **Manual de Usuario**; todo cambio de arquitectura o seguridad actualiza el **README**.

10.3 Herramientas recomendadas

Herramienta	Uso	Estado
EditorConfig	Formato base (indentación, UTF-8, fin de línea)	Configurado en frontend/.editorconfig
Prettier	Formato de TypeScript, HTML y CSS	Configurado en frontend/package.json
Angular Language Service	Autocompletado y errores en plantillas	Recomendado en .vscode/extensions.json

Herramienta	Uso	Estado
PHP Intelephense / PHP_CodeSniffer	Análisis y verificación de PSR-12	Recomendado (por configurar)
ESLint (angular-eslint)	Reglas de calidad de TypeScript y plantillas	Recomendado (por configurar)
PHPUnit	Pruebas del backend	Recomendado (por configurar)

11. Deuda técnica conocida

El proyecto creció rápido y algunas partes del código existente no cumplen todavía estos estándares. Esta lista guía las mejoras y evita repetir los mismos patrones en código nuevo. Se corrige de forma gradual, priorizando seguridad.

Prioridad	Hallazgo	Regla relacionada	Acción recomendada
Alta	Algunas acciones que modifican datos (`eliminar`, `activar`, `inactivar`) se envían por `GET`.	4.8	Migrarlas a `POST`/`DELETE` y validar el método en el controlador.
Alta	La consulta de usuarios devuelve la columna de contraseña (hash) al navegador.	4.6, 8.2	Excluir la columna `clave` de la consulta.
Alta	Algunos controladores devuelven `$e->getMessage()` al cliente.	4.8	Registrar con `error_log` y responder un mensaje genérico.
Alta	Las fotos de cédulas se sirven como archivos estáticos sin autenticación.	4.9	Servirlas por un endpoint que verifique el token.
Media	CORS permite cualquier origen (`*`) y se repite el encabezado en cada controlador.	4.10	Centralizar y restringir al dominio en producción.
Media	Se usa `any` de forma extensa (numerosos usos) en el frontend.	5.4	Crear interfaces por entidad y tipar los servicios.
Media	Componentes de más de 1000 líneas (créditos, caja, administrador).	2.2	Dividir en componentes y servicios más pequeños.
Media	Permisos de carpeta `0777` al crear directorios de carga.	4.9	Usar `0755`.
Baja	Uso de `alert()` y de `console.log` en varios componentes.	5.7	Reemplazar por notificaciones y eliminar los `console.log`.
Baja	Archivos PHP que cierran con `?>`.	4.3	Omitir la etiqueta de cierre.
Baja	Respuestas con las claves `estado` y `resultado` mezcladas.	4.8	Unificar en `estado` manteniendo compatibilidad.
Baja	No hay ESLint, PHPCS ni PHPUnit configurados.	9, 10.3	Incorporarlos de forma progresiva.

12. Anexos

12.1 Resumen de convenciones de nombres

Contexto	Archivos	Clases	Métodos / variables	Constantes
PHP	camelCase (clientesControlador.php)	PascalCase	camelCase	MAYÚSCULAS_SNAKE
TypeScript	kebab-case (gestion-pago.ts)	PascalCase	camelCase	MAYÚSCULAS o camelCase
Base de datos	rutalan.sql	tablas: snake_case plural	columnas: snake_case	ENUM en minúsculas
CSS	kebab-case (gastos.css)	clases: kebab-case	—	variables `--nombre`
Git	—	ramas: tipo/descripcion	commits: tipo(ámbito): texto	—

12.2 Lista de verificación rápida del Pull Request

- ☐ La rama y los commits siguen las convenciones.
- ☐ `ng build` y `php -l` pasan; `ng test` pasa si aplica.
- ☐ El código está formateado (Prettier / EditorConfig).
- ☐ Probé con rol administrador y cobrador, en móvil y escritorio.
- ☐ Toda acción de la API usa `requerirAuth()` y valida la entrada.
- ☐ Solo sentencias preparadas; ninguna contraseña ni secreto en el diff.
- ☐ No hay `console.log`, código comentado, `any` nuevo ni datos personales.
- ☐ Actualicé la documentación afectada.

12.3 Control de versiones del documento

Versión	Fecha	Descripción
1.0	Octubre de 2026	Primera edición de los estándares de codificación de Rutalan.

12.4 Documentos relacionados

- **README.md**: descripción general, instalación y seguridad de la API.
- **CONTRIBUTING.md**: guía de contribución, ramas, commits y Pull Requests.
- **Manual de Usuario Rutalan**: uso de la aplicación por rol.
- **ERS**: Formato de levantamiento de requerimientos IEEE 29148.
- **Diseño del proyecto**: Proceso de diseño del proyecto, desde el análisis de la situación actual del negocio hasta el modelado técnico del sistema.